

Nhóm 1: Docstring và hàm

Bài 1. Hàm tính tổng và trung bình

Viết hàm: `summarize(values)`

Trả về dictionary:

```
{  
    "count": ...,  
    "sum": ...,  
    "mean": ...,  
    "min": ...,  
    "max": ...  
}
```

Yêu cầu:

- Viết docstring mô tả đầy đủ tham số và kết quả.
- Phát sinh `ValueError` nếu values rỗng.
- Kiểm thử với list số nguyên và list số thực.

Bài 2. Phân loại chất lượng không khí

Viết hàm: `classify_pm25(value)`

Quy ước bài tập:

Khoảng PM2.5	Nhãn
< 12	Good
12–35	Moderate
> 35	High

Yêu cầu:

- Viết docstring theo dạng Parameters, Returns, Raises.
- Phát sinh `TypeError` nếu đầu vào không phải là số.
- Kiểm thử giá trị biên: 11.9, 12, 35, 35.1.

Bài 3. Chuẩn hóa dữ liệu

Viết hàm: `min_max_scale(values)`

Thực hiện chuẩn hóa theo công thức:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Yêu cầu:

- Trả về list giá trị mới.
- Không làm thay đổi list gốc.
- Xử lý trường hợp tất cả giá trị giống nhau.
- Viết docstring giải thích cách xử lý trường hợp `max(values) == min(values)`.

Bài 4. Hàm đọc bản ghi cảm biến

Viết hàm: `parse_sensor_record(record)`

Với dữ liệu đầu vào:

"TSH01,Tan Son Hoa,2026-09-09 08:00,31.5,29.2"

Kết quả mong đợi:

```
{
    "station_id": "TSH01",
    "station_name": "Tan Son Hoa",
    "timestamp": "2026-09-09 08:00",
    "pm25": 31.5,
    "temperature": 29.2
}
```

Yêu cầu:

- Docstring nêu rõ định dạng chuỗi đầu vào.
- Báo lỗi nếu bản ghi không gồm đủ 5 trường.
- Chuyển PM2.5 và nhiệt độ về float.

Nhóm 2: Lambda expressions

Bài 5. Sắp xếp dữ liệu nhiều tiêu chí

Cho dữ liệu:

```
stations = [  
    {"id": "A01", "name": "Tan Son Hoa", "pm25": 32.5, "temp": 30.1},  
    {"id": "A02", "name": "Thu Duc", "pm25": 18.2, "temp": 29.4},  
    {"id": "A03", "name": "Bien Hoa", "pm25": 41.8, "temp": 31.0},  
    {"id": "A04", "name": "Nha Be", "pm25": 32.5, "temp": 28.8}  
]
```

Yêu cầu dùng `sorted()` và `lambda` để:

- Sắp xếp PM2.5 giảm dần.
- Nếu PM2.5 bằng nhau thì sắp xếp nhiệt độ tăng dần.
- Trích ra list chỉ gồm tên trạm theo thứ tự đã sắp xếp.

Bài 6. Biến đổi đơn vị

Cho: `temperatures_c = [20, 25, 30, 35, 40]`

Dùng `map()` và `lambda` để:

- Đổi sang Fahrenheit.
- Làm tròn đến 1 chữ số thập phân.
- Tạo list nhãn "hot" nếu lớn hơn 32°C, ngược lại "normal".

Bài 7. Lọc dữ liệu hợp lệ

Cho dữ liệu: `raw_values = [12.5, -999, 18.0, None, 25.4, "error", 30.2, -1]`

Yêu cầu:

- Dùng `filter()` và `lambda` để lấy các phần tử là `int` hoặc `float`.
- Chỉ giữ giá trị lớn hơn hoặc bằng 0.
- Tính trung bình của dữ liệu hợp lệ.

Gợi ý: `isinstance(value, (int, float))`

Bài 8. Trích xuất thuộc tính

Cho:

```
records = [
```

("A01", "Tan Son Hoa", 32.5),

("A02", "Thu Duc", 18.2),

("A03", "Bien Hoa", 41.8)

]

Dùng lambda với map() để tạo: ["Tan Son Hoa", "Thu Duc", "Bien Hoa"]

Sau đó tạo list mới chứa các tuple: ("Tan Son Hoa", "High")

Trong đó "High" áp dụng cho PM2.5 lớn hơn 35.

Nhóm 3: Higher-order functions

Bài 9. Hàm áp dụng nhiều lần

Viết: `apply_n_times(func, value, n)`

Ví dụ: `apply_n_times(lambda x: x + 2, 1, 4)`

Kết quả: 9

Mở rộng:

- Nếu $n < 0$, phát sinh `ValueError`.
- Nếu $n == 0$, trả về giá trị ban đầu.

Bài 10. Data transformation pipeline

Viết: `run_pipeline(values, functions)`

Trong đó:

- `values` là list số.
- `functions` là list các hàm.
- Mỗi hàm được áp dụng lần lượt lên toàn bộ list.

Ví dụ:

```
functions = [  
    lambda x: x * 2,  
    lambda x: x + 1,  
    lambda x: round(x / 3, 2)  
]
```

Với: `values = [3, 6, 9]`

Yêu cầu:

- In kết quả sau từng bước.
- Không thay đổi list đầu vào.
- Viết docstring mô tả thứ tự áp dụng các hàm.

Bài 11. Chọn quy tắc phân loại

Viết: `classify_values(values, classifier)`

Trong đó `classifier` là một hàm nhận một giá trị và trả về nhãn.

Ví dụ:

```
def classify_temperature(temp):
```

```
    if temp < 20:
```

```
        return "cool"
```

```
    if temp <= 32:
```

```
        return "normal"
```

```
    return "hot"
```

Yêu cầu:

- Trả về dictionary dạng {value: label}.
- Thử với ít nhất 2 hàm phân loại khác nhau.
- Một hàm phân loại cho nhiệt độ, một hàm phân loại cho PM2.5.

Khoảng PM2.5	Nhãn
< 12	Good
12–35	Moderate
> 35	High

Bài 12. Bộ lọc tùy biến

Viết: `custom_filter(values, predicate)`

Không dùng hàm dựng sẵn `filter()`.

Yêu cầu:

- `predicate` là hàm nhận một giá trị và trả về True hoặc False.
- Trả về list chỉ gồm các giá trị thỏa điều kiện.
- Thử nghiệm với:
 - số chẵn,
 - số lớn hơn 25,
 - chuỗi có độ dài lớn hơn 5.

Nhóm 4: Function closure

Bài 13. Hàm kiểm tra ngưỡng

Viết: `make_threshold_checker(threshold)`

Ví dụ:

```
is_high = make_threshold_checker(35)
```

```
print(is_high(30))
```

```
print(is_high(40))
```

Kết quả: False

True

Bài mở rộng:

- Thêm tham số `inclusive=True`
- Nếu `inclusive=True`, sử dụng `>=`
- Nếu `inclusive=False`, sử dụng `>`

Bài 14. Hàm chuyển đổi đơn vị có cấu hình

Viết: `make_unit_converter(scale, offset)`

Hàm trả về một function thực hiện:

$$y = x \times scale + offset$$

Ví dụ: Celsius sang Fahrenheit:

```
c_to_f = make_unit_converter(9 / 5, 32)
```

```
print(c_to_f(25))
```

Bài 15. Bộ đếm sự kiện

Viết: `create_counter()`

Hàm trả về một closure có khả năng:

- Tăng bộ đếm lên 1 nếu không có đối số.
- Tăng bộ đếm theo số lượng chỉ định nếu có đối số.
- Trả về giá trị đếm hiện tại.

Ví dụ:

```
counter = create_counter()
```

```
print(counter())  
print(counter(3))  
print(counter())
```

Kết quả:

```
1  
4  
5
```

Nhóm 5: Wrapper và decorator

Bài 16. Decorator ghi log

Viết decorator: `log_call`

Yêu cầu in ra:

- Tên hàm.
- Tham số vị trí.
- Tham số có tên.
- Kết quả trả về.

Ví dụ định dạng:

[LOG] Function: `calculate_mean`

[LOG] args: `([10, 20, 30],)`

[LOG] kwargs: `{}`

[LOG] result: `20.0`

Yêu cầu kỹ thuật:

- Dùng `*args`, `**kwargs`.
- Dùng `functools.wraps`.

Bài 17. Decorator đo thời gian

Viết decorator: `timer`

Yêu cầu:

- Dùng `time.perf_counter()`.
- In tên hàm và thời gian thực thi.
- Trả về kết quả của hàm gốc.
- Dùng `@wraps`

Áp dụng cho hàm:

```
def sum_of_squares(n):  
    return sum(x ** 2 for x in range(n))
```

Test thử với `n = 100000` và `n = 1000000`.

Bài 18. Decorator kiểm tra dữ liệu rỗng

Viết decorator: `require_non_empty`

Yêu cầu:

- Áp dụng cho các hàm có đối số đầu tiên là list hoặc tuple.
- Nếu dữ liệu rỗng, phát sinh: `ValueError("Dữ liệu đầu vào không được rỗng")`
- Nếu dữ liệu hợp lệ, gọi hàm gốc.

Áp dụng cho:

`calculate_mean(values)`

`calculate_max(values)`

`calculate_min(values)`

Nhóm 6: Bài tập tổng hợp nâng cao

Bài 19. Xây dựng bộ phân tích dữ liệu quan trắc

Cho dữ liệu:

```
measurements = [  
    {"station": "A01", "pm25": 12.5, "temp": 28.2},  
    {"station": "A02", "pm25": 37.4, "temp": 31.0},  
    {"station": "A03", "pm25": 28.6, "temp": 30.4},  
    {"station": "A04", "pm25": 45.2, "temp": 32.1},  
    {"station": "A05", "pm25": 18.9, "temp": 29.5}  
]
```

Yêu cầu:

- Viết `extract_field(records, field)` dùng higher-order function hoặc lambda.
- Dùng `filter()` với lambda để lấy các trạm có PM2.5 lớn hơn 35.
- Viết closure `make_pm25_classifier(threshold)`.
- Viết decorator `log_call` cho hàm tạo báo cáo.
- Viết hàm có docstring:

`build_report(measurements, classifier)`

- Báo cáo cần có:
 - số lượng trạm,
 - trung bình PM2.5,
 - PM2.5 lớn nhất,
 - danh sách trạm vượt ngưỡng,
 - nhãn phân loại của từng trạm.

Bài 20. Xây dựng pipeline làm sạch dữ liệu

Cho dữ liệu thô:

```
raw_pm25 = [12.5, -999, None, 18.0, "NA", 35.2, -1, 42.6, 20.1]
```

Viết hệ thống pipeline gồm các hàm:

`remove_non_numeric(values)`

`remove_negative(values)`

`clip_upper_bound(values, upper=500)`

`round_values(values, digits=1)`

Yêu cầu:

- Mỗi hàm có docstring.
- Dùng higher-order function *run_pipeline*
- Dùng lambda hoặc filter() khi phù hợp.
- Viết decorator log_step để in:
 - tên bước xử lý,
 - số phần tử trước xử lý,
 - số phần tử sau xử lý.
- Không thay đổi list gốc.
- Trả về list sạch cuối cùng và thống kê cơ bản.

Ví dụ kết quả kỳ vọng:

`[12.5, 18.0, 35.2, 42.6, 20.1]`